
The MiniSQL Translator

Submitted By

Student Name	Student ID
Muhaiminul Islam Hasin	232-15-667
Md Mehrab Hossain	232-15-754

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSE314: Compiler Design Lab in the Computer Science and Engineering Department**



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

December 14, 2025

DECLARATION

We hereby declare that this **MiniSql Translator** lab project has been done by us under the supervision of **Chayti Saha, Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted to:

Chayti Saha

Lecturer,

Department of Computer Science and Engineering
Daffodil International University.Dhaka.

Submitted by

Muhaiminul Islam Hasin

Student ID: 232-15-667

Dept. of CSE, DIU

Md Mehrab Hossen

Student ID: 232-15-754

Dept. of CSE, DIU

Course & Program Outcome

The following course have course outcomes as following:

Table 1: Course Outcome Statements

CO's	Statements
CO1	Define and Relate classes, objects, members of the class, and relationships among them needed for solving specific problems.
CO2	Formulate knowledge of Compiler Design.
CO3	Analyze Diagram models to Present a specific problem
CO4	Develop solutions for real-world complex problems applying CD concepts while evaluating their effectiveness based on industry standards.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO1	C1, C2	KP3	EP1,EP3
CO2	PO2	C2	KP3	EP1,EP3
CO3	PO3	C4, A1	KP3	EP1,EP2
CO4	PO3	C3, C6, A3, P3	KP4	EP1,EP3

The mapping justification of this table is provided in section 4.3.1, 4.3.2 and 4.3.3

Table of Contents

Declaration	i
Course & Program Outcome	ii
1. Introduction	1
1.1 Introduction	1
1.2 Motivation	1
1.3 Objectives	2
1.4 Feasibility Study	3
1.5 Gap Analysis	5
1.6 Project Outcome	6
2. Proposed Methodology/Architecture	7
2.1 Requirement Analysis & Design Specification	7
2.1.1 Overview	13
2.1.2 Proposed Methodology/ System Design	14
2.2 Overall Project Plan	14
3. Implementation and Results	17
3.1 Implementation	17
3.2 Performance Analysis	24
3.3 Results and Discussion	24
4. Engineering Standards and Mapping	27
4.1 Impact on Society, Environment and Sustainability	27
4.1.1 Impact on Life	27
4.1.2 Impact on Society & Environment	27
4.1.3 Ethical Aspects	27
4.1.4 Sustainability Plan	28
4.2 Project Management and Team Work	28
4.3 Complex Engineering Problem	29
4.3.1 Mapping of Program Outcome	30
4.3.2 Complex Problem Solving	30
4.3.3 Engineering Activities	30
5. Conclusion	31
5.1 Summary	31
5.2 Limitation	31
5.3 Future Work	31
References	32

Chapter 1

Introduction

The MiniSQL project is a **simplified SQL parser and query handler** designed for students and beginners in database systems.

Purpose:

- 1.To provide a basic understanding of SQL language syntax and parsing
- 2.To demonstrate how SELECT-FROM-WHERE queries can be parsed and validated
- 3.To teach practical use of **Flex and Bison** in building a compiler-like parser

MiniSQL supports only **SELECT-FROM-WHERE** queries, making it a lightweight and educational project.

1.1 Introduction

SQL (Structured Query Language) is the standard language for relational databases. The MiniSQL project aims to develop a **basic SQL interpreter** that reads, parses, and validates user input queries. It allows the user to check the syntax of their queries and displays parsed information such as **table names, selected columns, and conditions** in a readable format.

1.2 Motivation

To gain hands-on experience in compiler design using **Flex** (lexer generator) and **Bison** (parser generator).

For me personally, solving this problem offers several benefits:

- **Skill Development:** It strengthens my understanding of programming in C, parsing techniques, and sql queries.
- **SQL Understanding :** To provide a practical demonstration of SQL syntax, including **SELECT, FROM, WHERE clauses, and operator precedence for AND/OR** conditions
- **Real-world Application:** It allows me to create a tool I can use in my studies and professional work without relying on commercial software or devices.
- **Extensibility & Customization:** I gain the flexibility to add new features (e.g., Generated/Computed Columns, CTEs (Common Table Expressions) based on my needs.
- **Contribution to Learning Tools:** I can share this project with classmates and educators to help them perform advanced calculations without additional cost.

In essence, the motivation stems from both **practical need**—to have a database instruction, customizable, and **academic interest**—to apply theoretical computer science concepts in a tangible, impactful way.

1.3 Objectives

The main objectives of the **Minisql Translator** project are:

1. Designing a **lexer** that identifies SQL keywords, identifiers, numbers, operators, and punctuation.
2. Creating a **parser** that validates SQL queries according to the grammar rules defined for SELECT-FROM-WHERE.
3. Handling **simple logical expressions** in WHERE clauses using AND, OR, and parentheses.
4. Providing **user-friendly output** showing table names, selected columns, and applied conditions.
5. Ensuring **memory safety** by proper allocation and deallocation of dynamically stored strings

1.4 Feasibility Study

Technical Feasibility:

1. MiniSQL requires only standard C libraries along with Flex and Bison. No heavy dependencies or database engines are required.
2. The parser can run on Windows, Linux, or macOS with GCC or compatible compilers.

Operational Feasibility:

1. The system is user-friendly and requires minimal input from users.
2. No installation of database software is needed; it simulates SQL query validation.

Economic Feasibility:

1. Low development cost since it uses open-source tools (Flex, Bison, GCC).
2. No recurring costs are required for maintenance.

1.4 Gap Analysis

While MiniSQL successfully parses basic SELECT-FROM-WHERE queries, it **does not support**:

- INSERT, UPDATE, DELETE, or complex JOIN queries
- Aggregate functions (SUM, AVG, COUNT)
- Nested queries beyond a single WHERE clause
- Full database storage and query execution

The gap indicates opportunities for **future expansion**, such as building a full-fledged SQL interpreter with database integration.

1.5 Project Outcome

The expected outcomes of the MiniSQL project are:

- A working **SQL parser** capable of reading and validating SELECT-FROM-WHERE queries.
- Demonstration of **Flex-Bison integration** for tokenizing and parsing language syntax.
- Display of **structured query information** (table, columns, conditions) in a readable format.
- Hands-on learning experience in **compiler design**, memory management, and parsing strategies.
- A foundation for future enhancements towards a **complete SQL interpreter or query engine**.

Overall, the outcome will be a **ready-to-use, open-source, sql database translation tool** that addresses current gaps in cloud technology, particularly for users who prefer or require terminal-based solutions.

Chapter 2

Proposed Methodology/Architecture

2.1 Requirement Analysis & Design Specification

A. Requirement Analysis

The **requirement analysis** phase of the MiniSQL project involves understanding the functional and non-functional requirements of a simplified SQL parser and query validator.

1. Functional Requirements

1. The system should parse SELECT-FROM-WHERE queries.
2. Recognize SQL keywords: SELECT, FROM, WHERE, AND, OR.
3. Support operators: =, <, >, <=, >=, !=.
4. Identify **table names**, **column names**, and **conditional expressions**.
5. Handle logical expressions with **parentheses**.
6. Output parsed information in a structured and readable format.

2. Non-Functional Requirements

The system should:

1. Lightweight, with minimal dependencies (requires only C, Flex, Bison).
2. Memory-efficient, ensuring proper allocation and deallocation.
3. User-friendly input and output interface.
4. Platform-independent and compilable on Windows, Linux, and macOS.

B. Design Specification

Lexer (Flex):

- Reads user input.
- Breaks the input into **tokens** (keywords, identifiers, operators, numbers, punctuation).
- Sends tokens to the parser.

Parser (Bison):

- Receives tokens from the lexer.
- Validates syntax according to the defined **SQL grammar rules**.
- Constructs structured representations of the query (table name, columns, WHERE clause).
- Handles operator precedence and nested conditions.

Memory Management:

- Dynamically allocates memory for identifiers and expressions.
- Frees memory to prevent leaks.

Output Module:

- Displays parsed results clearly: table name, selected columns, conditions.

2.1.2 Proposed Methodology / System Design

The **system design methodology** follows a top-down approach, focusing on parsing and validation of SQL queries:

Input Handling:

- Accept SQL query strings from user input.
- Trim and preprocess input if necessary.

Lexical Analysis (Lexer):

- Identify tokens (keywords, identifiers, numbers, operators).
- Return tokens to parser with associated values (yylval).

Syntax Analysis (Parser):

- Define grammar rules for SELECT-FROM-WHERE queries.
- Handle optional WHERE clause with logical expressions.
- Construct string representations for table, columns, and conditions.

Error Handling:

- Detect syntax errors and report them clearly.
- Handle invalid tokens and missing clauses gracefully.

Output Generation:

- Display structured information in user-friendly format.

System Architecture Diagram (Conceptual):

User Input



Lexer (Flex) → Token Stream



Parser (Bison) → Syntax Validation & Parsing



Structured Query Output (Table, Columns, Conditions)

2.2 Overall Project Plan

The MiniSQL project plan includes the following phases:

Requirement Gathering:

- Define scope (SELECT-FROM-WHERE support only).
- Identify functional and non-functional requirements.

System Design:

- Create lexer and parser specifications.
- Design grammar rules and memory management strategies.

Implementation:

- Develop Flex lexer file (mini.l).
- Develop Bison parser file (mini.y).
- Integrate lexer and parser using GCC compilation.

Testing:

- Validate queries for correct syntax.
- Test operator precedence and logical expressions.
- Check memory handling for dynamic allocations.

Documentation and Reporting:

- Prepare project report with introduction, methodology, design, testing, and outcomes.
- Include example queries and execution results.

Timeline (Suggested):

Phase	Duration
Requirement Analysis	1 week
System Design	1 week
Implementation	2 weeks
Testing & Debugging	1 week
Documentation	1 week

Chapter 3

Implementation and Results

3.1 Implementation

The implementation phase of the **MiniSQL** project focuses on developing a working **SQL parser** capable of recognizing and validating **SELECT-FROM-WHERE** queries. The main components implemented are:

1. Lexer Implementation (Flex)

- The **Flex lexer (mini.l)** reads user input and breaks it into tokens.
- **Recognized tokens include:**
 - SQL keywords: SELECT, FROM, WHERE, AND, OR
 - Operators: =, <, >, <=, >=, !=
 - Symbols: *, ,, ,, (,)
 - Identifiers (column/table names) and numeric literals

- **Example rule for identifiers:**

```
[a-zA-Z_][a-zA-Z0-9_]* { yylval.sval = strdup(yytext); return ID; }
```

- **Numbers are converted from string to integer:**

-

```
[0-9]+ { yylval.ival = atoi(yytext); return NUMBER; }
```

2. Parser Implementation (Bison)

- The **Bison parser (mini.y)** defines grammar rules for SELECT-FROM-WHERE queries.
- Handles optional WHERE clause and nested logical expressions using recursion.
- Supports operator precedence:
 - AND has higher precedence than OR.
- Generates a structured representation of the query:
 - current_table → table name
 - selected_columns → comma-separated column names
 - where_clause → combined string of conditions

- Memory management is handled carefully with malloc, strdup, and free to avoid leaks.

3. Integration

- Lexer and parser are integrated using standard GCC compilation:

```
bison -d mini.y  
flex mini.l  
gcc mini.tab.c lex.yy.c -o minisql
```

3.1 Performance Analysis

Although MiniSQL is primarily an educational tool, the following **performance observations** were made:

1. **Parsing Efficiency:**

- Lexical analysis using Flex is fast for queries with up to 20–30 tokens.
- Recursive parsing in Bison handles nested WHERE clauses efficiently for moderate query complexity.

2. **Memory Usage:**

- Dynamic allocation for identifiers and condition strings is managed safely.
- Memory leaks are prevented by freeing allocated strings after use.

3. **Scalability:**

- MiniSQL is not designed for large databases or high query loads.
- It can easily handle small queries with multiple columns and conditions for educational purposes.

4. **Error Detection:**

- Syntax errors are detected immediately.
- Invalid tokens or malformed queries result in clear error messages.

3.2 Results and Discussion

1. **Results:**

- The project successfully parses and validates **SELECT-FROM-WHERE** queries.
- Displays parsed table names, selected columns, and WHERE conditions.
- Correctly handles logical expressions with AND, OR, and parentheses.
- Syntax errors are reported clearly and immediately.

```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Version 10.0.26100.6725]
(c) Microsoft Corporation. All rights reserved.

F:\project>bison -d minisql.y
F:\project>flex minisql.l
F:\project>gcc lex.yy.c minisql.tab.c -o minisql
F:\project>minisql.exe
MiniSQL - SELECT-FROM-WHERE Parser
Example queries:
  SELECT name, age FROM students WHERE age >= 18;
  SELECT * FROM teachers;
Enter your SQL query:
|
```

Fig 01: Flex & Bison Program Run on
Commant Prompt , Show Input For User
- Interface.

```
Enter your SQL query:

SELECT * FROM students;

Valid SQL query.
Table      : students
Columns    : *
WHERE      : (none)
```

Fig 02: Sample input Run and Compile
show output.

```
Enter your SQL query:

SELECT id, FROM students;
Syntax error: syntax error

F:\project>
```

Fig 03: Show syntax error Output.

2.Discussion:

- The project demonstrates how **Flex and Bison** can be used to implement a compiler-like parser for SQL.
- Recursive grammar allows parsing of nested conditions, while operator precedence ensures correct evaluation order.
- Memory management using `strdup`, `malloc`, and `free` prevents memory leaks.
- Limitations:
 - Does not support INSERT, UPDATE, DELETE, or JOIN queries.
 - Does not connect to a real database.
 - Limited to SELECT-FROM-WHERE queries only.
- Potential improvements:
 - Extend grammar to support additional SQL commands.
 - Integrate a small database backend to execute queries.
 - Implement optimization for handling very large queries or multiple queries in batch.

Chapter 4

Engineering Standards and Mapping

4.1 Impact on Society, Environment and Sustainability

A. Societal Impacts

By teaching SQL parsing and database fundamentals, it equips students and developers with skills applicable in IT, software engineering, and data management. Encourages a culture of **structured learning and experimentation** in computer science.

B. Environmental Impact

The project is lightweight and fully digital; it has **minimal environmental footprint**. Does not require server infrastructure or physical database hardware for basic operations, reducing energy consumption.

C. Ethical Aspects

- MiniSQL emphasizes **ethical use of software**:
- Encourages proper data handling practices (e.g., understanding queries without directly accessing sensitive data).
- Promotes **transparent programming** and reproducible results.
- Acts as a **learning tool**, not a system for unauthorized database access.

D. Sustainability Considerations

Code is **open-source and reusable**, ensuring long-term educational use. Designed for scalability; future developers can extend grammar to more SQL features without major redesign. Documentation and modular code structure support **maintenance and knowledge transfer**.

4.2 Project Management and Team Work/Team Contribution

4.2 Project Management and Team Work / Team Contribution

Member Name	Student ID	Contribution Summary
Muhaiminul Islam Hasin	232-15-667	Lexer and parser development, Efficient debugging and feature implementation.
Md Mehrab Hossain	232-15-754	Regular meetings to align on grammar rules, memory handling, and testing, Development of communication and coordination skills , essential in engineering projects

Team Collaboration Approach

- Joint Decision Making: All major design and coding decisions were discussed collectively via group meetings.
- Code Review: Each module was peer-reviewed by at least one other member before integration.
- Testing: Every member contributed to functional and error-handling tests.
- Final Report & Presentation: Prepared collaboratively, with content reviewed and approved by the entire team.

This balanced allocation ensured that each member contributed both in development and in supporting tasks, resulting in a well-coordinated and successfully delivered project.

4.3 Complex Engineering Problem

4.3.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provided solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program Outcomes

PO's	Justification
PO1	We used fundamental compiler design knowledge to understand the problem and apply correct concepts like tokens, parsing, and syntax rules.
PO2	We analyzed the input, identified errors, and applied logical steps to check validity and solve the problem.
PO3	We designed and implemented a working compiler component using tools like Flex and Bison to generate the correct output.

4.3.2 Complex Problem Solving

In this section, provide a mapping with problem solving categories. For each mapping add subsections to put rationale (Use Table 4.2). For P1, you need to put another mapping with Chapter 4. Engineering Standards and Mapping 4.3. Complex Engineering Problem Knowledge profile and rational thereof.

Table 4.2: Mapping with complex problem solving.

EP1 Dept of Knowledge	EP2 Range of Conflicting Requirements	EP3 Depth of Analysis	EP4 Familiarity of Issues	EP5 Extent of Applicable Codes	EP6 Extent of Stakeholder Involvement	EP7 Interdependence
√	√					

EP1 – Depth of Knowledge:

The problem needs good understanding of compiler design topics like lexical analysis, parsing, and grammar rules.

EP2 – Range of Conflicting Requirements:

The solution must handle different input patterns correctly while avoiding errors, so it requires balancing accuracy and flexibility.

4.3.3 Engineering Activities

In this section, provide a mapping with engineering activities. For each mapping add subsections to put rationale (Use Table 4.3).

Table 4.3: Mapping with complex engineering activities.

EA1 Range of resources	EA2 Level of Interaction	EA3 Innovation	EA4 Consequences for society and environment	EA5 Familiarity
√	√			

EA1 – Range of Resources:

The task uses different technical tools like Flex, Bison, code editors, and compiler concepts to complete the work.

EA2 – Level of Interaction:

The activity requires interacting with multiple components such as tokens, grammar rules, and parsing steps to produce the final output.

Chapter 5

Conclusion

5.1 Summary

The MiniSQL project successfully demonstrates the design and implementation of a **simplified SQL parser** for SELECT-FROM-WHERE queries. Key achievements include:

- Development of a **lexer using Flex** to tokenize SQL keywords, identifiers, numbers, and operators.
- Creation of a **parser using Bison** to validate query syntax and handle nested logical expressions.
- Correct implementation of **operator precedence** (AND > OR) and support for parentheses in conditions.
- Memory-safe handling of dynamically allocated strings using `malloc`, `strdup`, and `free`.
- Structured output showing table names, selected columns, and WHERE clause conditions.

Overall, the project serves as an **educational tool** for understanding SQL syntax, compiler design principles, and practical use of Flex-Bison integration.

5.2 Limitation

- Supports only **SELECT-FROM-WHERE** queries; other SQL commands like INSERT, UPDATE, DELETE, JOIN, and aggregate functions are not implemented.
- Does not interact with a **real database**; it only parses and validates queries.
- Handling of very large or complex queries may be limited due to static buffer sizes and string concatenation.
- No optimization for multiple queries or batch processing.

5.3 Future Work

Extended SQL Support:

Implement additional SQL commands: INSERT, UPDATE, DELETE, JOIN, and aggregation functions.

Database Integration:

Connect the parser to a real database engine to execute validated queries.

User Interface Enhancement:

Develop a GUI or web interface for easier query input and output display.

Performance Optimization:

Improve memory handling for large queries and batch processing.

Advanced Query Features:

Support nested queries, subqueries, and more complex logical expressions.

